24MCAL191 – PROGRAMMING LAB

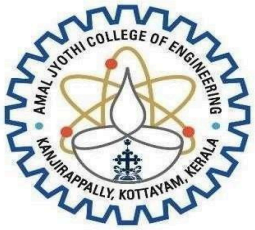
Report Submitted By

JOSEPH MATHEW

AJC25MCA-2041

In Partial fulfilment for the Award of the Degree Of

MASTER OF COMPUTER APPLICATIONS
(MCA TWO YEARS)
[Accredited by NBA]
APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY



AMAL JYOTHI
COLLEGE OF ENGINEERING
AUTONOMOUS
KANJIRAPPALLY

[Affiliated to APJ Abdul Kalam Technological University, Kerala. Approved by AICTE, Accredited by NAAC. Koovappally, Kanjirapally, Kottayam, Kerala – 686518

2025-2026

DEPARTMENT OF COMPUTER APPLICATIONS

AMAL JYOTHI COLLEGE OF ENGINEERING

KANJIRAPALLY

CERTIFICATE

This is to certify that the lab report, "**24MCAL191 - PROGRAMMIMG LAB"** is the bona fide work of **JOSEPH MATHEW AJC25MCA-2041** in partial fulfilment of the requirements for the award of the Degree of Master of Computer Applications under APJ Abdul Kalam Technological University, during the year **2025-26.**

Mr. Jobin T J

Lab In- Charge

Dr. Bijimol T K

Head of the Department

Internal Examiner

External Examiner

Lab Record

Microproject

AMAL JYOTHI
COLLEGE OF ENGINEERING
KANJIRAPPALLY, KOTTAYAM, INDIA 686 518

Course Code	Course Name	Syllabus Year
24MCAL191	PROGRAMMIMG LAB	2024

VISION

To promote an academic and research environment conducive for innovation centric technical education.

MISSION

● MS1 - Provide foundations and advanced technical education in both theoretical and applied Computer Applications in-line with Industry demands.
● MS2 - Create highly skilled computer professionals capable of designing and innovating real life solutions.
● MS3 - Sustain an academic environment conducive to research and teaching focused to generate up-skilled professionals with ethical values.
● MS4 - Promote entrepreneurial initiatives and innovations capable of bridging and contributing with sustainable, socially relevant technology solutions.

COURSE OUTCOME

CO	Outcome
CO1	Understands basics of Python Programming language including input/output functions, operators, basic and collection data types.
CO2	Implement decision making, looping constructs and functions.
CO3	Design modules and packages - built in and user defined packages.
CO4	Implement object-oriented programming and exception handling.
CO5	Create files and form regular expressions for effective search operations on strings and files.

CONTENTS

SL NO.	LIST OF LAB EXPERIMENTS/EXERCISES	DATE	CO	PAGE NO
8	Print all colors that are present in color_list1 but not in color_list2.	29-09-2025	CO2	9
9	Create a single string separated by a space from two strings after swapping the character at index 1.	06-10-2025	CO2	10
10	Sort a dictionary in both ascending and descending order.	06-10-2025	CO2	11
11	Merge two dictionaries into one.	06-10-2025	CO2	12
12	Find the greatest common divisor (GCD) of two numbers.	13-10-2025	CO2	13
13	From a list of integers, create a new list after removing all even numbers.	13-10-2025	CO2	14
14	Generate a Fibonacci series of N terms.	13-10-2025	CO2	15
15	Find the sum of all items in a list using function.	10-11-2025	CO3	16
16	Count the frequency of each character in a string using function.	10-11-2025	CO3	17
17	Add ‘ing’ to the end of a given string. If it already ends with ‘ing’, add ‘ly’ instead using function	10-11-2025	CO3	18
18	Write lambda functions to find the areas of a square, a rectangle, and a triangle using function	10-11-2025	CO3	19
19	Create a Rectangle class with attributes length and breadth, along with methods to compute area and perimeter. Compare two rectangle objects based on their areas.	17-11-2025	CO4	20

20	Create a BankAccount class with attributes: account number, name, account type, and balance. Write a constructor and methods to deposit and withdraw money from the bank account.	17-11-2025	CO4	22
SL NO.	**LIST OF LAB EXPERIMENTS/EXERCISES**	**DATE**	**CO**	**PAGE NO**
21	Create a Time class with private attributes hour, minute, and second, and overload the + operator to add two time objects.	17-11-2025	C04	23
22	Write a Python program to read a file line by line and store each line in a list.	15-12-2025	CO5	25
23	Write a Python program to read specific columns of a CSV file and print the contents.	15-12-2025	CO5	26
24	Write a Python program to write a dictionary to a CSV file, then read back the file and display its contents.	15-12-2025	CO5	27

AMAL JYOTHI
COLLEGE OF ENGINEERING
KANJIRAPPALLY, KOTTAYAM, INDIA 686 518

Experiment No: 1

Aim: Generate a list containing only the positive numbers from a given list of integers.

CO1: Understands basics of Python Programming language including input/output functions, operators, basic and collection data types.

Program code:

```
numbers = list(map(int, input("Enter integers separated by space: ").split()))
positive = []
for n in numbers:
if n > 0:
positive.append(n)
print(positive)
```

Result: The program was executed. The result was obtained successfully. Thus CO1 was obtained.

Output:

```
Enter integers separated by space: 2 4 6 -1 -3 0
[2, 4, 6]
```

AMAL JYOTHI
COLLEGE OF ENGINEERING
KANJIRAPPALLY, KOTTAYAM, INDIA 686 518

Experiment No: 2

Aim: Compute the square of N numbers.

CO1: Understands basics of Python Programming language including input/output functions, operators, basic and collection data types.

Program code:

```
n = int(input("Enter number of elements: "))
numbers = []

print("Enter the numbers:")
for i in range(n):
    num = int(input())
    numbers.append(num)

squares = []
for num in numbers:
    squares.append(num * num)

print("Squares of the numbers:", squares)
```

Result: The program was executed. The result was obtained successfully. Thus CO1 was obtained.

Output:

```
Enter number of elements: 4
Enter the numbers:
2
3
4
5
Squares of the numbers: [4, 9, 16, 25]
```

AMAL JYOTHI
COLLEGE OF ENGINEERING
KANJIRAPPALLY, KOTTAYAM, INDIA 686 518

Experiment No: 3

Aim: Form a list containing all vowels selected from a given word.

CO2: Implement decision making, looping constructs and functions.

Program code:

```
word = input("Enter a word: ")

vowels = ['a', 'e', 'i', 'o', 'u']
vowel_list = []

for ch in word.lower():
    if ch in vowels:
        vowel_list.append(ch)

print("Vowels in the word:", vowel_list)
```

Result: The program was executed. The result was obtained successfully. Thus CO2 was obtained.

Output:

```
Enter a word: Education
Vowels in the word: ['e', 'u', 'a', 'i', 'o']
```

Experiment No: 4

Aim: Store a list of first names and count the total occurrences of the letter ‘a’ in the list.

CO2: Implement decision making, looping constructs and functions.

Program code:

```
n = int(input("Enter number of names: "))
names = []

print("Enter the first names:")
for i in range(n):
    name = input()
    names.append(name)

count = 0
for name in names:
    count += name.lower().count('a')

print("Total occurrences of 'a':", count)
```

Result: The program was executed. The result was obtained successfully. Thus CO2 was obtained.

Output:

```
Enter number of names: 3
Enter the first names:
thomas
jacob
mathew
Total occurrences of 'a': 3
```

Experiment No: 5

Aim: Enter two lists of integers and check:

a) whether the lists are of equal length,

b) whether their sums are equal, and

c) whether any values are common to both lists.

CO2: Implement decision making, looping constructs and functions.

Program code:

```
list1 = list(map(int, input("Enter first list: ").split()))
list2 = list(map(int, input("Enter second list: ").split()))

# a) Equal length
if len(list1) == len(list2):
print("Same length")
else:
print("Different length")

# b) Equal sum
if sum(list1) == sum(list2):
print("Same sum")
else:
print("Different sum")

# c) Common values
common = []
for x in list1:
if x in list2 and x not in common:
common.append(x)

if common:
print("Common values:", common)
```

```
else:
print("No common values")
```

Result: The program was executed. The result was obtained successfully. Thus CO2 was obtained.

Output:

```
Enter first list: 2 4 5 6 7
Enter second list: 6 4 2 1 3
Same length
Different sum
Common values: [2, 4, 6]
```

Experiment No: 6

Aim: Create a new string in which all occurrences of the first character are replaced with '$', except the first character itself.

CO2: Implement decision making, looping constructs and functions.

Program code:

```
s = input("Enter a string: ")

first = s[0]
new_string = first

for ch in s[1:]:
if ch == first:
new_string += '$'
else:
new_string += ch

print("New string:", new_string)
```

Result: The program was executed. The result was obtained successfully. Thus CO2 was obtained.

Output:

```
Enter a string: restart
New string: resta$t
```

Experiment No: 7

Aim: Create a string from a given string by exchanging its first and last characters.

CO2: Implement decision making, looping constructs and functions.

Program code:

```
string = input("Enter a string: ")

if len(string) < 2:
   print("New string:", string)
else:
   new_string = string[-1] + string[1:-1] + string[0]
   print("New string:", new_string)
```

Result: The program was executed. The result was obtained successfully. Thus CO2 was obtained.

Output:

```
Enter a string: joseph
New string: hosepj
```

Experiment No: 8

Aim: Print all colors that are present in color_list1 but not in color_list2.

CO2: Implement decision making, looping constructs and functions.

Program code:

```
color_list1 = ["Red", "Green", "White", "Black"]
color_list2 = ["Green", "White"]

for color in color_list1:
    if color not in color_list2:
        print(color)
```

Result: The program was executed. The result was obtained successfully. Thus CO2 was obtained.

Output:

```
Enter colors of list 1: red blue yellow green
Enter colors of list 2: red blue
Colors in list1 but not in list2:
yellow
green
```

Experiment No: 9

Aim: Create a single string separated by a space from two strings after swapping the character at index 1.

CO2: Implement decision making, looping constructs and functions.

Program code:

```
s1 = input("Enter first string: ")
s2 = input("Enter second string: ")
if len(s1) > 1 and len(s2) > 1:
s1_new = s1[0] + s2[1] + s1[2:]
s2_new = s2[0] + s1[1] + s2[2:]
else:
s1_new = s1
s2_new = s2
result = s1_new + " " + s2_new
print("Result:", result)
```

Result: The program was executed. The result was obtained successfully. Thus CO2 was obtained.

Output:

```
Enter first string: joseph
Enter second string: thomas
Result: jhseph toomas
```

Experiment No: 10

Aim: Sort a dictionary in both ascending and descending order.

CO2: Implement decision making, looping constructs and functions.

Program code:

```
my_dict = {
   'b': 7,
   'a': 1,
   'd': 3,
   'c': 8
}

asc_dict = dict(sorted(my_dict.items()))
desc_dict = dict(sorted(my_dict.items(), reverse=True))

print("Ascending order:", asc_dict)
print("Descending order:", desc_dict)
```

Result: The program was executed. The result was obtained successfully. Thus CO2 was obtained.

Output:

```
Ascending order: {'a': 1, 'b': 7, 'c': 8, 'd': 3}
Descending order: {'d': 3, 'c': 8, 'b': 7, 'a': 1}
```

Experiment No: 11

Aim: Merge two dictionaries into one.

CO2: Implement decision making, looping constructs and functions.

Program code:

```
d1 = eval(input("Enter first dictionary: "))
d2 = eval(input("Enter second dictionary: "))

d1.update(d2)
print("Merged dictionary:", d1)
```

Result: The program was executed. The result was obtained successfully. Thus CO2 was obtained.

Output:

```
Enter first dictionary: {'a':3, 'f':4}
Enter second dictionary: {'e':1, 'h':5}
Merged dictionary: {'a': 3, 'f': 4, 'e': 1, 'h': 5}
```

Experiment No: 12

Aim: Find the greatest common divisor (GCD) of two numbers.

CO2: Implement decision making, looping constructs and functions.

Program code:

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))

while b != 0:
    a, b = b, a % b
print("GCD:", a)
```

Result: The program was executed. The result was obtained successfully. Thus CO2 was obtained.

Output:

```
Enter first number: 12
Enter second number: 24
GCD: 12
```

AMAL JYOTHI
COLLEGE OF ENGINEERING
KANJIRAPPALLY, KOTTAYAM, INDIA 686 518

Experiment No: 13

Aim: From a list of integers, create a new list after removing all even numbers.

CO2: Implement decision making, looping constructs and functions.

Program code:

```
numbers = list(map(int, input("Enter integers separated by space: ").split()))

odd_numbers = []

for num in numbers:
if num % 2 != 0:
odd_numbers.append(num)

print("List after removing even numbers:", odd_numbers)
```

Result: The program was executed. The result was obtained successfully. Thus CO2 was obtained.

Output:

```
Enter integers separated by space: 2 3 7 5 6 1 9
List after removing even numbers: [3, 7, 5, 1, 9]
```

Experiment No: 14

Aim: Generate a Fibonacci series of N terms.

CO2: Implement decision making, looping constructs and functions.

Program code:

```
N = 11
a, b = 0, 1
fib_series = []

for _ in range(N):
    fib_series.append(a)
    a, b = b, a + b

print(fib_series)
```

Result: The program was executed. The result was obtained successfully. Thus CO2 was obtained.

Output:

```
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55]
```

AMAL JYOTHI
COLLEGE OF ENGINEERING
KANJIRAPPALLY, KOTTAYAM, INDIA 686 518

Experiment No: 15

Aim: Find the sum of all items in a list using function.

CO3: Design modules and packages - built in and user defined packages

Program code:

```
def sum_of_list(lst):
    total = 0
    for num in lst:
        total += num
    return total

numbers = list(map(int, input("Enter the numbers: ").split()))
print("Sum of all items:", sum_of_list(numbers))
```

Result: The program was executed. The result was obtained successfully. Thus CO3 was obtained.

Output:

```
Enter the numbers: 3 4 5 6 7
Sum of all items: 25
```

Experiment No: 16

Aim: Count the frequency of each character in a string using function.

CO3: Design modules and packages - built in and user defined packages

Program code:

```
def char_frequency(s):
    freq = {}
    for ch in s:
        if ch in freq:
            freq[ch] += 1
        else:
            freq[ch] = 1
    return freq

text = str(input("enter the string:"))
result = char_frequency(text)
print(result)
```

Result: The program was executed. The result was obtained successfully. Thus CO3 was obtained.

Output:

```
enter the string:python
{'p': 1, 'y': 1, 't': 1, 'h': 1, 'o': 1, 'n': 1}
```

Experiment No: 17

Aim: Add ‘ing’ to the end of a given string. If it already ends with ‘ing’, add ‘ly’ instead using function

CO3: Design modules and packages - built in and user defined packages

Program code:

```
def add_suffix(s):
    if s.endswith("ing"):
        return s + "ly"
    else:
        return s + "ing"

string = input("Enter a string: ")
result = add_suffix(string)

print("Result:", result)
```

Result: The program was executed. The result was obtained successfully. Thus CO3 was obtained.

Output:

```
Enter a string: soft
Result: softing
```

```
Enter a string: softing
Result: softingly
```

AMAL JYOTHI
COLLEGE OF ENGINEERING
KANJIRAPPALLY, KOTTAYAM, INDIA 686 518

Experiment No: 18

Aim: Write lambda functions to find the areas of a square, a rectangle, and a triangle using function

CO3: Design modules and packages - built in and user defined packages

Program code:

```
area_square = lambda side: side * side
area_rectangle = lambda length, breadth: length * breadth
area_triangle = lambda base, height: 0.5 * base * height


s = float(input("Enter side of square: "))
print("Area of square:", area_square(s))

l = float(input("Enter length of rectangle: "))
b = float(input("Enter breadth of rectangle: "))
print("Area of rectangle:", area_rectangle(l, b))

base = float(input("Enter base of triangle: "))
h = float(input("Enter height of triangle: "))
print("Area of triangle:", area_triangle(base, h))
```

Result: The program was executed. The result was obtained successfully. Thus CO3 was obtained.

Output:

```
Enter side of square: 4
Area of square: 16.0
Enter length of rectangle: 2
Enter breadth of rectangle: 4
Area of rectangle: 8.0
Enter base of triangle: 3
Enter height of triangle: 5
Area of triangle: 7.5
```

Experiment No: 19

Aim: Create a Rectangle class with attributes length and breadth,along with methods to compute area and perimeter.Compare two rectangle objects based on their areas.

CO4: Implement object-oriented programming and exception handling.

Program code:

```
class Rectangle:
    def __init__(self, length, breadth):
        self.length = length
        self.breadth = breadth

    def area(self):
        return self.length * self.breadth
    def perimeter(self):
        return 2 * (self.length + self.breadth)


r1 = Rectangle(10,10)
print(f"Area of Rectangle: {r1.area()}")
print(f"Perimeter ofRectangle Perimeter:{r1.perimeter()}")
r2 = Rectangle(10, 20)
print(f"Area of Rectangle: {r2.area()}")
print(f"Perimeter ofRectangle Perimeter:{r2.perimeter()}")

if r1.area() > r2.area():

    print("\nRectangle 1 has larger area")
elif r1.area() < r2.area():
    print("\nRectangle 2 has larger area")
else:
    print("\nBoth rectangles have equal area")
```

Result: The program was executed. The result was obtained successfully. Thus CO4 was obtained.

Output:

```
Area of Rectangle: 100
Perimeter ofRectangle Perimeter:40
Area of Rectangle: 200
Perimeter ofRectangle Perimeter:60

Rectangle 2 has larger area
```

AMAL JYOTHI
COLLEGE OF ENGINEERING
KANJIRAPPALLY, KOTTAYAM, INDIA 686 518

Experiment No: 20

Aim: Create a BankAccount class with attributes: accountnumber, name, account type, and balance. Write constructor and methods to deposit and withdraw money from the bank account

CO4: Implement object-oriented programming and exception handling.

Program code:

```
class BankAccount:
    def __init__(self):
        self.balance = 10000


    def deposit(self, amount):
        self.balance += amount
        return self.balance
    def withdraw(self, amount):
        self.balance -= amount
        return self.balance

a = BankAccount()
print(a.deposit(500))
print(a.withdraw(1500))
print(a.deposit(1000))
```

Result: The program was executed. The result was obtained successfully. Thus CO4 was obtained.

Output:

```
10500
9000
10000
```

.

Experiment No: 21

Aim: Create a Time class with private attributes hour, minute, and second, and overload the + operator to add two time objects.

CO4: Implement object-oriented programming and exception handling.

Program code:

```
class Time:
    def __init__(self, h, m, s):
        self.__hour = h
        self.__minute = m
        self.__second = s

    def __add__(self, other):
        s = self.__second + other.__second
        m = self.__minute + other.__minute
        h = self.__hour + other.__hour

        if s >= 60:
            m += s // 60
            s = s % 60

        if m >= 60:
            h += m // 60
            m = m % 60

        return Time(h, m, s)

    def display(self):
        print(f"{self.__hour:02d}:{self.__minute:02d}:{self.__second:02d}")

print("Enter Time 1")
h1 = int(input("Hour: "))
m1 = int(input("Minute: "))
s1 = int(input("Second: "))
```

```
print("Enter Time 2")
h2 = int(input("Hour: "))
m2 = int(input("Minute: "))
s2 = int(input("Second: "))

t1 = Time(h1, m1, s1)
t2 = Time(h2, m2, s2)

t3 = t1 + t2

print("Sum of Time:")
t3.display()
```

Result: The program was executed. The result was obtained successfully. Thus CO4 was obtained.

Output:

```
Enter Time 1
Hour: 2
Minute: 30
Second: 24
Enter Time 2
Hour: 3
Minute: 23
Second: 10
Sum of Time:
05:53:34
```

Experiment No: 22

Aim: Write a Python program to read a file line by line and store each line in a list.

CO5: Create files and form regular expressions for effective search operations on strings and files.

Program code:

```
lines = []

with open("sample.txt", "r") as file:
    for line in file:
        lines.append(line.strip())

print("Lines stored in list:")
print(lines)
```

Result: The program was executed. The result was obtained successfully. Thus CO5 was obtained.

Output:

```
Lines stored in list:
['Apple', 'Banana', 'Mango']
```

AMAL JYOTHI
COLLEGE OF ENGINEERING
KANJIRAPPALLY, KOTTAYAM, INDIA 686 518

Experiment No: 23

Aim: Write a Python program to read specific columns of a CSV file and print the contents.

CO5: Create files and form regular expressions for effective search operations on strings and files.

Program code:

Data.csv

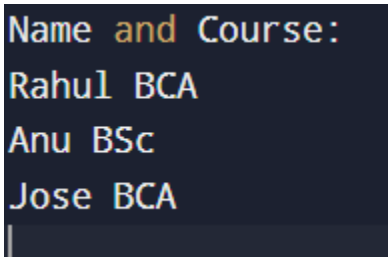
```
ID,Name,Age,Course
1,Rahul,20,BCA
2,Anu,21,BSc
3,Jose,22,BCA
```

```
import csv

with open("data.csv", "r", newline="") as file:
    reader = csv.reader(file)
    header = next(reader)
    print("Name and Course:")
    for row in reader:
        if len(row) >= 4:
            print(row[1], row[3])
```

Result: The program was executed. The result was obtained successfully. Thus CO5 was obtained.

Output:



Experiment No: 24

Aim: Write a Python program to write a dictionary to a CSV file,then read back the file and display its contents.
CO5: Create files and form regular expressions for effective search operations on strings and files.

Program code:

```
import csv

data = {
   "Name": "Alice",
   "Age": 25,
   "City": "New York",
   "Profession": "Engineer"
}

filename = "data.csv"

with open(filename, mode="w", newline="") as file:
   writer = csv.writer(file)
   writer.writerow(["Key", "Value"])
   for key, value in data.items():
      writer.writerow([key, value])

print("Dictionary written to CSV file.\n")

print("Contents of CSV file:")
with open(filename, mode="r") as file:
   reader = csv.reader(file)
   for row in reader:
      print(row)
```

Result: The program was executed. The result was obtained successfully. Thus CO5 was obtained.

Output:

```
Contents of CSV file:
['Key', 'Value']
['Name', 'Alice']
['Age', '25']
['City', 'New York']
['Profession', 'Engineer']
```